

MCS 320 Project Two : Code for Algorithmic Differentiation due Wednesday 22 July 2026, at 2pm.

The goal of the project is to generate code to apply algorithmic differentiation to polynomials in several variables. The project description was derived from a Jupyter notebook with a SageMath kernel, posted at the course web site. The best way to start working on the project is to execute the posted notebook.

1. Symbolic Gradient of a Product

We start with computing the gradient of the product of n variables: $\prod_{i=1}^n x_i$.

Symbolic differentiation gives us the expressions for the gradient of this product. For example, for $n = 8$, the list below contains the product and all its partial derivatives:

```
[x1*x2*x3*x4*x5*x6*x7*x8, x2*x3*x4*x5*x6*x7*x8, x1*x3*x4*x5*x6*x7*x8,
 x1*x2*x4*x5*x6*x7*x8, x1*x2*x3*x5*x6*x7*x8, x1*x2*x3*x4*x6*x7*x8,
 x1*x2*x3*x4*x5*x7*x8, x1*x2*x3*x4*x5*x6*x8, x1*x2*x3*x4*x5*x6*x7]
```

Observe the repetitiveness of the arithmetical operations: the sequence of the same k products occurs multiple times. Using the straightforward symbolic computation of the gradient of a product requires a number of multiplications that grows proportional to n^2 , quadratically in the number of variables.

2. Reverse Mode of Algorithmic Differentiation

In the reverse mode of algorithmic differentiation, extra variables accumulate the forward products:

$$f_2 = x_1 \star x_2, \quad f_i = f_{i-1} \star x_i, \quad \text{for } i = 3, \dots, n,$$

so f_n then holds the value of the product of the n variables, and f_{n-1} is the last component of the gradient. The backward products are computed as

$$b_{n-1} = x_n \star x_{n-1}, \quad b_i = b_{i+1} \star x_i, \quad \text{for } i = n-2, \dots, 2,$$

and we observe that b_2 equals the first component of the gradient. The other components of the gradient are computed via the cross products:

$$c_1 = x_1 \star b_3, \quad c_i = f_i \star b_{i+2}, \quad \text{for } i = 2, \dots, n-3, \quad c_{n-2} = f_{n-2} \star x_n.$$

The number of multiplications equals $3n - 5$. The cost to evaluate and differentiate a product of n variables grows linearly in n , as n increases.

For $n = 8$, the formulas are coded in a Python function below.

```
def gradprod(x1, x2, x3, x4, x5, x6, x7, x8):
    """
    Returns the function values and the gradient
    of the product of the values in x.
    """
    f2 = x1*x2
    f3 = f2*x3
    f4 = f3*x4
    f5 = f4*x5
```

```

f6 = f5*x6
f7 = f6*x7
f8 = f7*x8
b7 = x8*x7
b6 = b7*x6
b5 = b6*x5
b4 = b5*x4
b3 = b4*x3
b2 = b3*x2
c1 = x1*b3
c2 = f2*b4
c3 = f3*b5
c4 = f4*b6
c5 = f5*b7
c6 = f6*x8
return (f8, b2, c1, c2, c3, c4, c5, c6, f7)

```

Comparing the output of `timeit()` of this function to the plain evaluation of the symbolic formulas for the gradient, we notice a drop in units: from microseconds to nanoseconds.

3. Code Generation

If the definition of the above Python function is stored in a triple quoted string (with the documentation string removed), then running `exec()` on that string defines the function. We can use this construction to build string representations of functions to evaluate the gradient for any number of variables.

Assignment One. Take the formulas for the forward, backward, and cross products and write a function `G(n)` to generate a string for any number of variables, generalizing the code for $n = 8$. The input to `G` is n , the number of variables. The function `G` returns a string defining a Python function `GP` to evaluate a product of n variables. For $n = 8$, the output `GP` should be the same as the above function. In particular, work only with local variables to store the accumulated products. Do not use lists or any other composite data structures.

Time the generated code for $n = 8, 16, 32, 64, 128, 256$ and format the times in a table. Do you observe the $3n$ progression of the times as n increases?

Assignment Two. Consider monomials in n variables which are products of a selection of k variables:

$$\prod_{j=1}^k x_{i_j}$$

for a set of indices $\{i_1, i_2, \dots, i_k\} \subset \{1, 2, \dots, n\}$.

Extend the code to generate `GP` with an extra argument: the list of indices $i_1, i_2, \dots, i_k$.

Time the execution for $k = 8, 16, 32, 64, 128, 256$, where $n = 2k$, for random selections of k indices. Format the times in a table. Do you observe the $3k$ progression of the times as k increases?

Assignment Three. Consider a polynomial of m monomials in n variables, where every monomial is a product of variables:

$$p = \sum_{i=1}^m c_i \prod_{j=1}^{k_i} x_{i_j},$$

where the coefficients c_i are nonzero integer numbers.

Extend the code to generate `GP` to evaluate and differentiate p . The coefficients and monomials are given in two lists, respectively a list of integers and a list of tuples with the indices.

Generate random polynomials where $m = n$, and where the number of variables in each monomial is $n/2$, for $n = 8, 16, 32, 64, 128, 256$. Compare the evaluate a random integers with the symbolic substitution. Format the times of the symbolic substitution and the algorithmic differentiation in a table. Describe your findings.

4. The Deadline is Wednesday 22 July, at 2pm.

Upload your answer to gradescope at the latest on Wednesday 22 July, before 2pm.

The solution consists of one single notebook, organized according to the assignments. Mark the start of each solution to an assignment using a heading in a cell. Your notebook should run from top to bottom as a program without errors. Apply proper formatting in your notebook so it reads like a technical report if you would print it. Document the execution cells with complete sentences, properly formatted in markdown cells.

You may (not must) work in pairs for this project. A pair consists of two, not three or more. If you decide to work in a pair, then you must send me an email with the name of your partner and with the email address of your partner in the copy of the email, before 2pm on Friday 17 July. If working in a pair, then only one Jupyter notebook should be submitted.

If you have questions, concerns, or difficulties, feel free to contact me for help.